

EXPERIMENT 5

Name-Shraeyaa Dhaigude

Class-D20A

Roll no-22

Aim:Deploying a Voting/Ballot Smart Contract

Theory:

Relevance of require statements in solidity programs

In Solidity, the require statement acts as a guard condition within functions. It ensures that only valid inputs or authorized users can execute certain parts of the code. If the condition inside require is not satisfied, the function execution stops immediately, and all state changes made during that transaction are reverted to their original state. This rollback mechanism ensures that invalid transactions do not corrupt blockchain data.

For example, in a Voting Smart Contract, require can be used to check:

- Whether the person calling the function has the right to vote
(require(voters[msg.sender].weight > 0, "Has no right to vote");).
- Whether a voter has already voted before allowing them to vote again.
- Whether the function caller is the chairperson before granting voting rights.

Thus, require statements enforce security, correctness, and reliability in smart contracts. They prevent unauthorized access, eliminate invalid operations, and maintain the integrity of the voting process. Additionally, developers can attach error messages to require, which improves debugging and makes contract interaction clearer for users.

Keywords: mapping, storage, and memory

Mapping

A mapping is a special data structure in Solidity that links keys to values, similar to a hash table. Its syntax is:

```
mapping(keyType => valueType)
```

For example:

```
mapping(address => Voter) public voters;
```

Here, each address (Ethereum account) is mapped to a Voter structure. Mappings are very useful in contracts like Ballot, where it is necessary to associate each voter with their data (such as whether they have voted and which proposal they selected).

Unlike arrays:

- Mappings do not have a length property.
- They cannot be iterated over directly.
- They are highly gas-efficient for lookups.

Therefore, mappings are ideal for secure and efficient voter data management.

Storage

In Solidity, storage refers to the permanent memory of the contract stored on the blockchain. Variables declared at the contract level are stored in storage by default.

Key characteristics:

- Data is persistent across transactions.
- It remains available unless explicitly modified.
- Writing to storage consumes gas and is relatively expensive.
- For example, a voter's information saved in the voters mapping remains available throughout the contract's lifecycle. Storage is essential for maintaining the state of the voting system, such as voter records and proposal vote counts.

Memory

In contrast, memory is temporary storage used only during the execution of a function call. When the function finishes execution, data stored in memory is discarded.

Key characteristics:

- Temporary lifetime.
- Cheaper than storage.
- Used for local variables, function arguments, and intermediate calculations.
- For example, when handling proposal names passed into a function or performing temporary computations, memory is used instead of storage.
- Thus, a smart contract developer must carefully choose between storage and memory to balance cost efficiency and functionality.

Why bytes32 instead of string?

In earlier implementations of the Ballot contract, bytes32 was used for proposal names instead of string. The reason lies mainly in gas efficiency and simplicity.

bytes32 is a fixed-size data type that always stores exactly 32 bytes. This makes:

- Storage is simpler.
- Comparison operations faster.
- Gas consumption is lower.

However, it limits proposal names to 32 characters, reducing flexibility.

On the other hand, string is a dynamically sized data type. It can store text of variable length, making it more user-friendly and readable. However:

- It requires more complex handling inside the Ethereum Virtual Machine (EVM).
- It consumes more gas.
- String comparisons are more expensive.

Modern Ballot contract implementations often use string for better usability, while earlier versions preferred bytes32 for performance optimization.

In summary:

- bytes32 is preferred when performance and gas efficiency are priorities.
- string is preferred when readability and flexibility are more important.

Code-

```
// SPDX-License-Identifier: GPL-3.0
```

```
pragma solidity >=0.7.0 <0.9.0;
```

```
contract Ballot {

    struct Voter {
        uint weight;
        bool voted;
        address delegate;
        uint vote;
    }

    struct Proposal {
        string name;
        uint voteCount;
    }

    address public chairperson;

    mapping(address => Voter) public voters;

    Proposal[] public proposals;

    constructor(string[] memory proposalNames) {
        chairperson = msg.sender;
        voters[chairperson].weight = 1;

        for (uint i = 0; i < proposalNames.length; i++) {
            proposals.push(Proposal({
                name: proposalNames[i],
                voteCount: 0
            }));
        }
    }
}
```

```

function giveRightToVote(address voter) external {

    require(
        msg.sender == chairperson,
        "Only chairperson can give right to vote."
    );

    require(
        !voters[voter].voted,
        "The voter already voted."
    );

    require(
        voters[voter].weight == 0,
        "Voter already has the right to vote."
    );

    voters[voter].weight = 1;
}

function delegate(address to) external {

    Voter storage sender = voters[msg.sender];

    require(sender.weight != 0, "You have no right to
vote");
    require(!sender.voted, "You already voted.");
    require(to != msg.sender, "Self-delegation is
disallowed.");

    while (voters[to].delegate != address(0)) {
        to = voters[to].delegate;

        require(to != msg.sender, "Found loop in
delegation.");
    }

    Voter storage delegate_ = voters[to];

    require(delegate_.weight >= 1);

    sender.voted = true;
    sender.delegate = to;

    if (delegate_.voted) {

        proposals[delegate_.vote].voteCount +=
sender.weight;
    }
}

```

```

    } else {

        delegate_.weight += sender.weight;

    }
}

function vote(uint proposal) external {

    Voter storage sender = voters[msg.sender];

    require(sender.weight != 0, "Has no right to vote");
    require(!sender.voted, "Already voted.");

    sender.voted = true;
    sender.vote = proposal;

    proposals[proposal].voteCount += sender.weight;

}

function winningProposal() public view returns (uint
winningProposal_) {

    uint winningVoteCount = 0;

    for (uint p = 0; p < proposals.length; p++) {

        if (proposals[p].voteCount > winningVoteCount) {

            winningVoteCount = proposals[p].voteCount;
            winningProposal_ = p;

        }

    }

}

function winnerName() external view returns (string memory
winnerName_) {

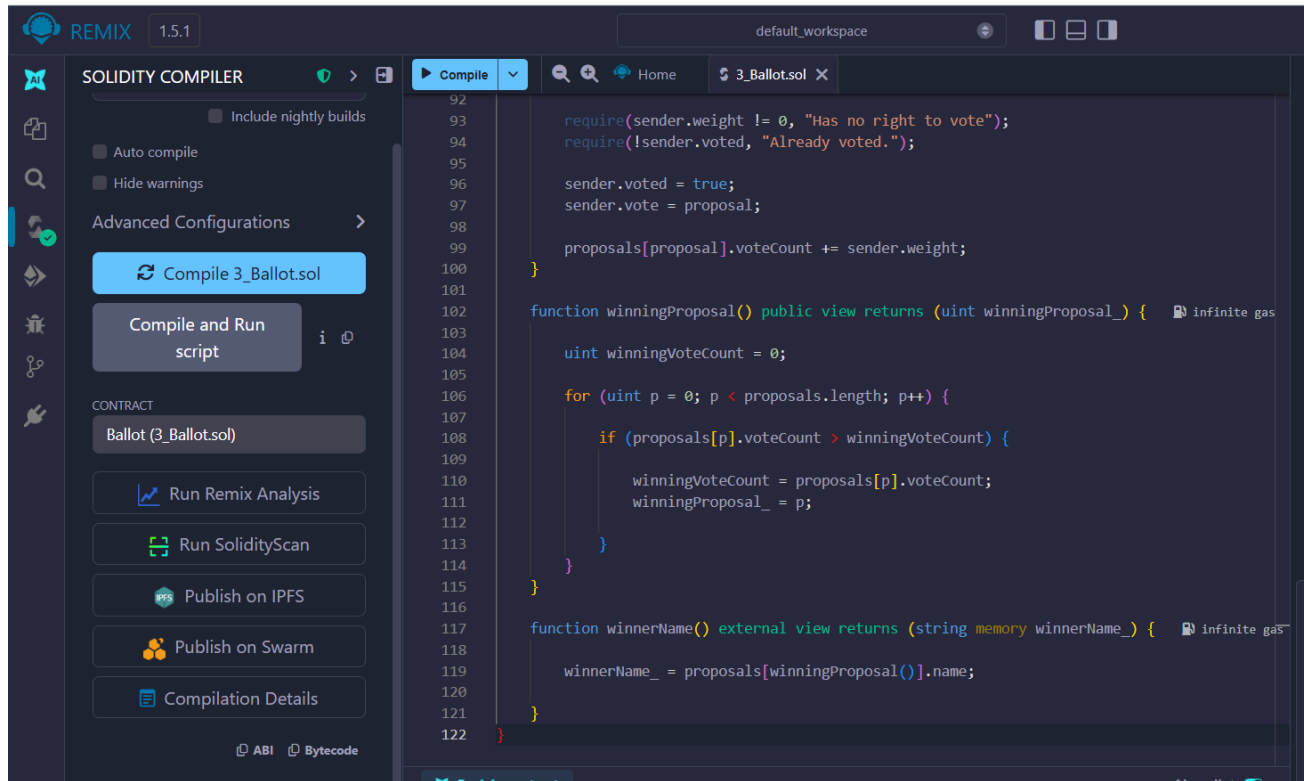
    winnerName_ = proposals[winningProposal()].name;

}
}

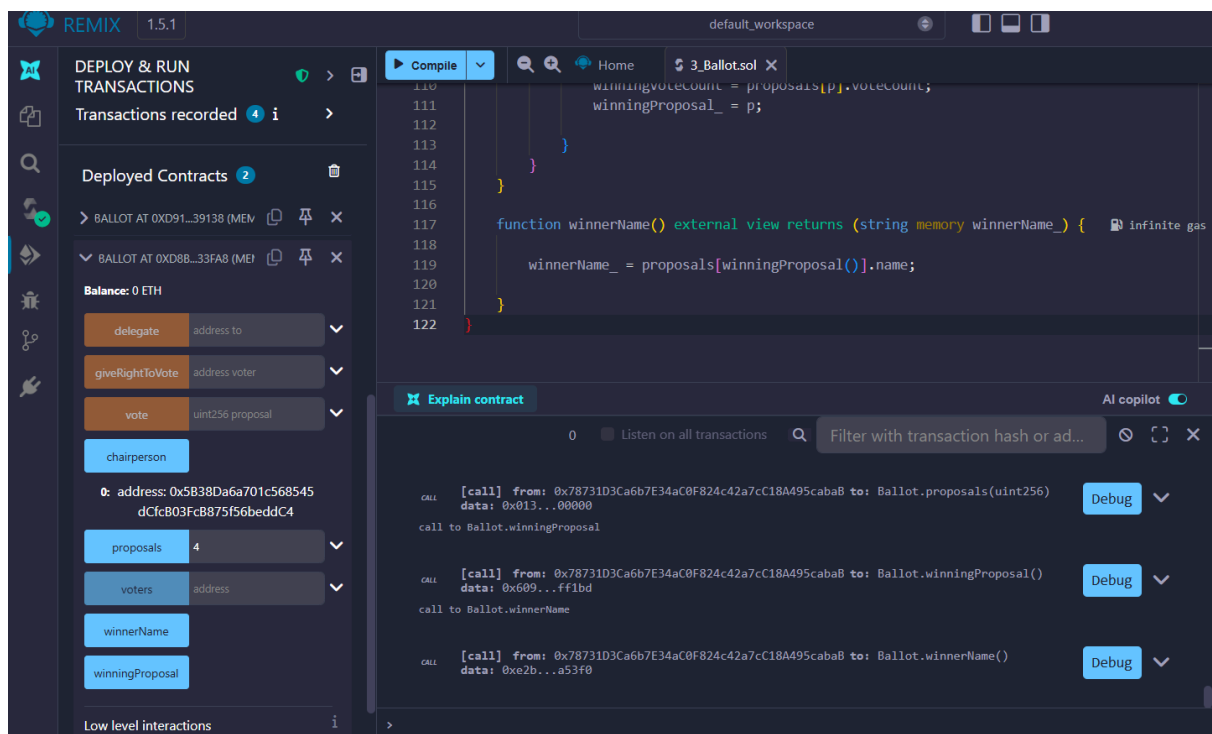
```

Output-

1)Compiled Ballot.sol contract



2)Deploying and running of contract



3) Loading the Proposal Candidate's Names (string)

The screenshot shows the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' sidebar is visible. It shows the environment set to 'Remix VM (Osaka)', the account '0x4B0...4D2dB (99.999999999)', and the gas limit set to '3000000'. The contract 'Ballot - contracts/3_Ballot.sol' is selected. The 'Deploy' button is highlighted. Below the sidebar, the 'Transactions recorded' section shows 27 transactions.

The main editor displays the Solidity code for '3_Ballot.sol'. The code includes a loop to iterate over proposals and a function 'winnerName()' that returns the name of the winning proposal candidate.

```
105
106
107
108     for (uint p = 0; p < proposals.length; p++) {
109
110         if (proposals[p].voteCount > winningVoteCount) {
111
112             winningVoteCount = proposals[p].voteCount;
113             winningProposal_ = p;
114         }
115     }
116
117     function winnerName() external view returns (string memory winnerName_) {
118         infinite gas
119         winnerName_ = proposals[winningProposal()].name;
```

The 'Explain contract' panel at the bottom shows the execution details of the contract. It lists several transactions, including the creation of the Ballot contract and the execution of the 'winnerName()' function. The 'winnerName()' function call is highlighted, showing the return value '0 wei data: 0x608...00000'.

4) Viewing the details of the proposal candidate

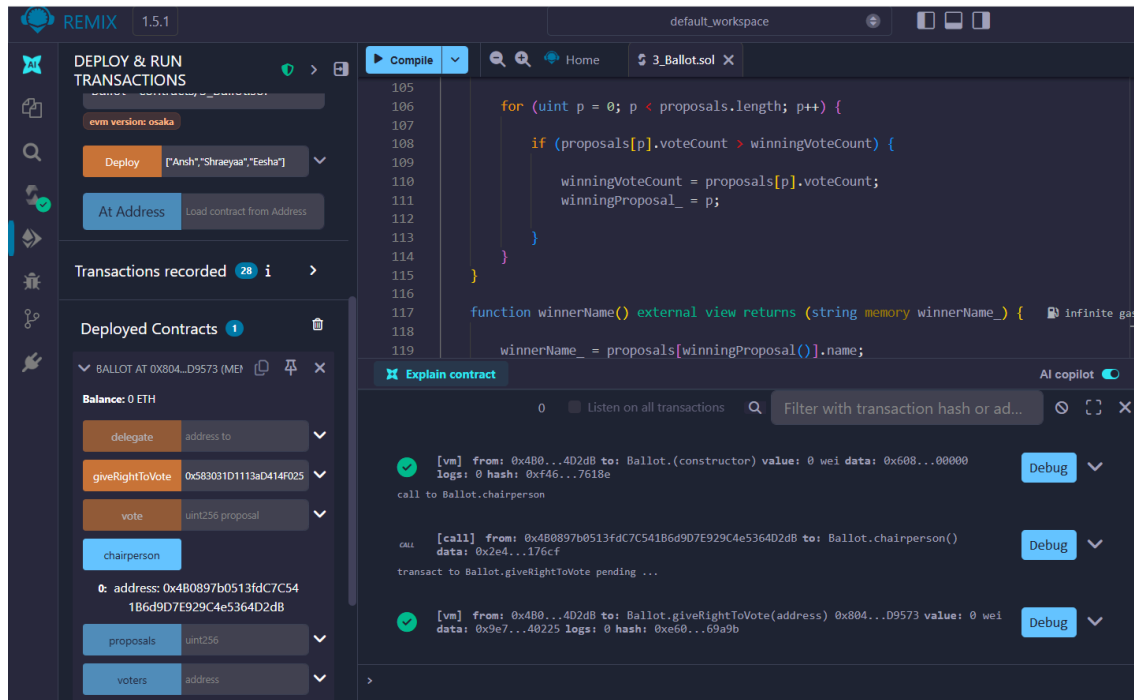
The screenshot shows the 'Ballot AT 0X3B2...826F1 (MEM)' details view. It displays the contract's balance as '0 ETH'. Below the balance, there are several input fields for interacting with the contract:

- delegate**: address to
- giveRightToVote**: address voter
- vote**: uint256 proposal
- chairperson**: 0: address: 0xcC6f9A5f0c7942A2caE5774d9961D33f846b5257
- proposals**: 1
- voters**: address
- winnerName**
- winningProposal**

The 'proposals' field is expanded, showing the details of the proposal candidate:

- 0: string: name Shraeyaa**
- 1: uint256: voteCount 0**

5) Giving right to vote other than the chairperson



The screenshot shows the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' panel is open, showing a list of deployed contracts. The main editor displays the Solidity code for the Ballot contract. The bottom panel shows the 'Explain contract' view, which lists transactions and their details.

```
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

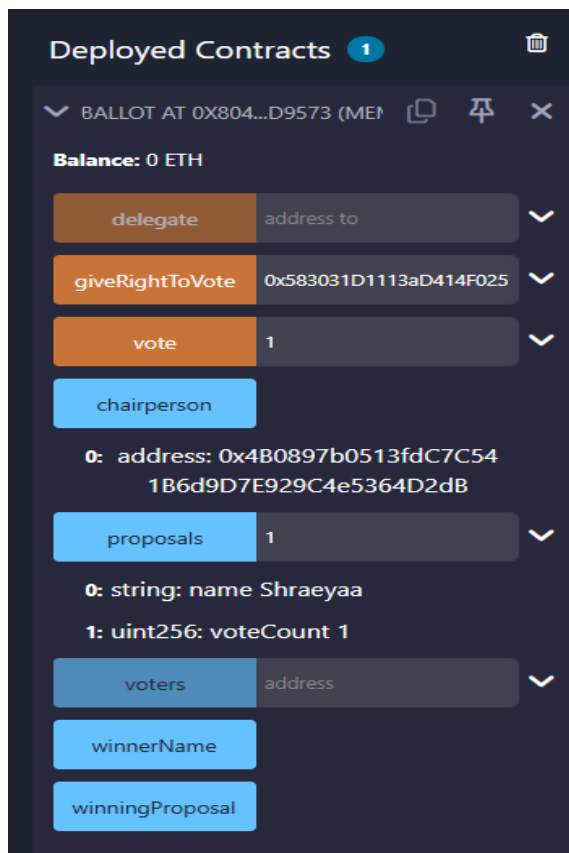
Deployed Contracts

- BALLOT AT 0x804...D9573 (ME)
- Balance: 0 ETH
- delegate address to
- giveRightToVote 0x583031D1113aD414F025
- vote uint256 proposal
- chairperson
- 0: address: 0x4B0897b0513fdC7C541B6d9D7E929C4e5364D2dB
- proposals uint256
- voters address
- winnerName
- winningProposal

Explain contract

- [vm] from: 0x4B0...4D2dB to: Ballot.(constructor) value: 0 wei data: 0x608...00000 logs: 0xf46...7618e
- call to Ballot.chairperson
- [call] from: 0x4B0897b0513fdC7C541B6d9D7E929C4e5364D2dB to: Ballot.chairperson() data: 0x2e4...176cf
- transact to Ballot.giveRightToVote pending ...
- [vm] from: 0x4B0...4D2dB to: Ballot.giveRightToVote(address) 0x804...D9573 value: 0 wei data: 0x9e7...40225 logs: 0xe60...69a9b

6) Selecting the account other than the chairperson to vote and then writing the proposal candidate's index to vote.



The screenshot shows the 'Deployed Contracts' panel in the Remix IDE. It displays the details of the 'BALLOT AT 0x804...D9573 (ME)' contract. The panel shows the contract's balance, methods, and state variables.

Deployed Contracts

- BALLOT AT 0x804...D9573 (ME)
- Balance: 0 ETH
- delegate address to
- giveRightToVote 0x583031D1113aD414F025
- vote 1
- chairperson
- 0: address: 0x4B0897b0513fdC7C541B6d9D7E929C4e5364D2dB
- proposals 1
- 0: string: name Shraeyaa
- 1: uint256: voteCount 1
- voters address
- winnerName
- winningProposal

7) We can view the Voter's information

DEPLOY & RUN

TRANSACTIONS

BALLOT AT 0X804...D9573 (MEM)

Balance: 0 ETH

delegate

address to

giveRightToVote

0x583031D1113aD414F025

vote

1

chairperson

0: address: 0x4B0897b0513fdC7C54
1B6d9D7E929C4e5364D2dB

proposals

1

0: string: name Shraeyaa

1: uint256: voteCount 1

voters

0x583031D1113aD414F025

0: uint256: weight 1

1: bool: voted false

2: address: delegate 0x000000000000
0000000000000000000000000000000000
0

3: uint256: vote 0

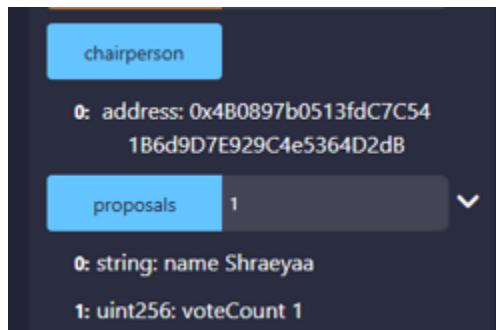
8) Selecting the account of the delegator and then writing the address of the voter to be delegated.

The screenshot displays the REMIX IDE interface with the following components:

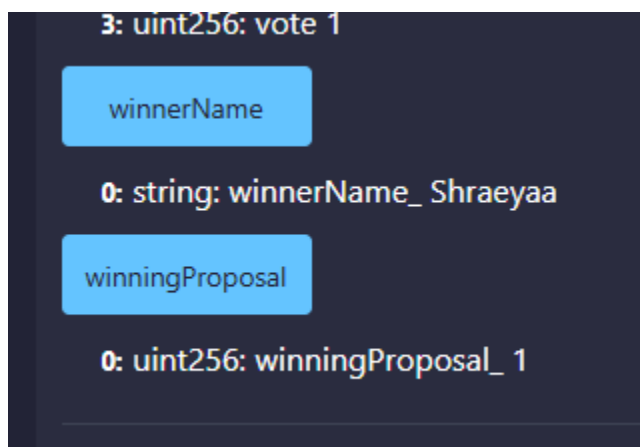
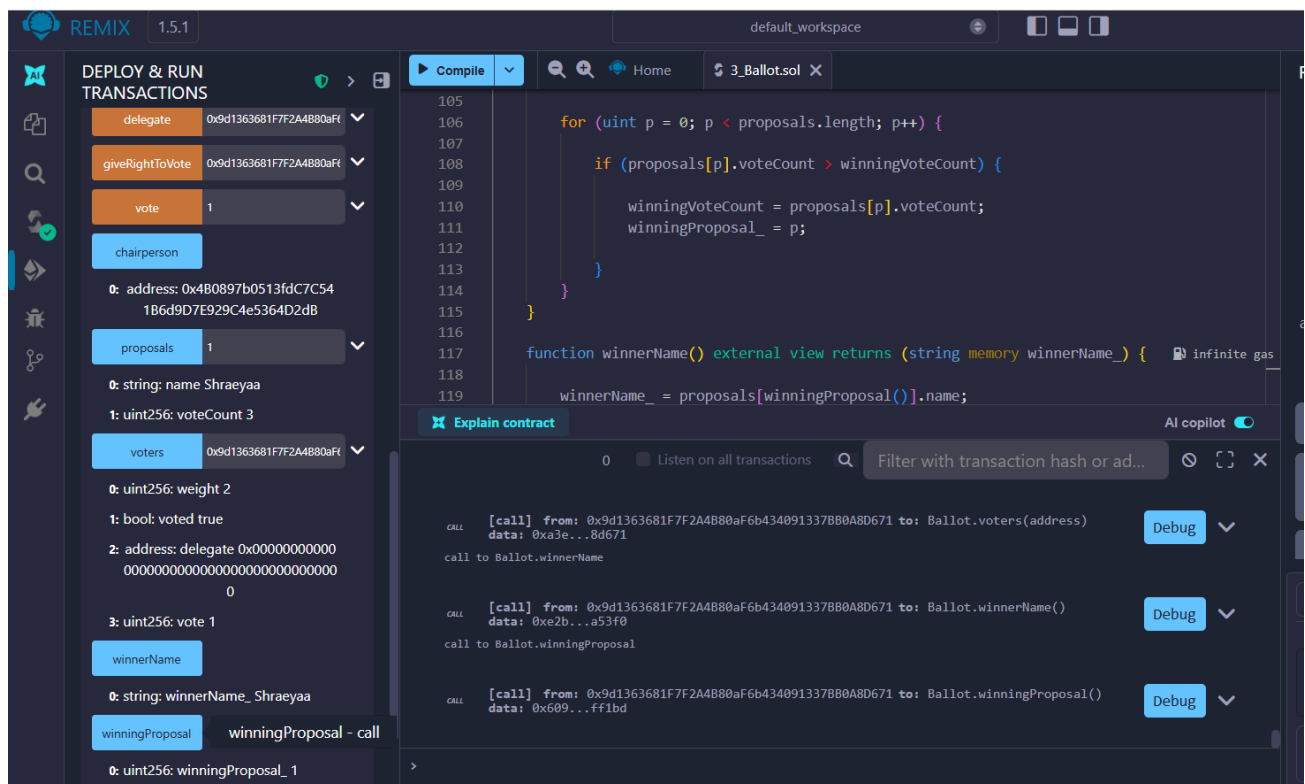
- Left Panel (DEPLOY & RUN TRANSACTIONS):**
 - Deployed Contracts:** A contract named "BALLOT AT 0x804...D9573 (ME)" is listed with a balance of 0 ETH.
 - Transaction Inputs:**
 - delegate:** 0x9d1363681f7f2A4B80aF...
 - giveRightToVote:** 0x9d1363681f7f2A4B80aF...
 - vote:** 1
 - chairperson:** (empty field)
 - proposals:** 1
 - string: name Shraeyaa**
 - uint256: voteCount 1**
 - voters:** 0x583031D11113aD414F025
 - uint256: weight 1**
 - bool: voted false**
 - address: delegate 0x00**
- Top Panel (Code Editor):** Shows Solidity code for `3_Ballot.sol`. The code includes a `for` loop to iterate through proposals and a `winnerName()` function.

```
105
106     for (uint p = 0; p < proposals.length; p++) {
107
108         if (proposals[p].voteCount > winningVoteCount) {
109
110             winningVoteCount = proposals[p].voteCount;
111             winningProposal_ = p;
112
113         }
114     }
115
116
117     function winnerName() external view returns (string memory winnerName_) {
118         // Infinite gas
119         winnerName_ = proposals[winningProposal()].name;
```
- Bottom Panel (Transaction Logs):** Displays three successful transactions with their respective data and logs.
 - Transaction 1:** [vm] from: 0x4B0...4D2dB to: Ballot.giveRightToVote(address) 0x804...D9573 value: 0 wei data: 0x9e7...8d671 logs: 0 hash: 0x2f8...672bb. Log: transact to Ballot.delegate pending ...
 - Transaction 2:** [vm] from: 0xdB8...92148 to: Ballot.delegate(address) 0x804...D9573 value: 0 wei data: 0x5c1...8d671 logs: 0 hash: 0xca6...2c6d6. Log: transact to Ballot.vote pending ...
 - Transaction 3:** [vm] from: 0x9d1...8D671 to: Ballot.vote(uint256) 0x804...D9573 value: 0 wei data: 0x012...00001 logs: 0 hash: 0x31f...0e307.

9) Proposal candidates information before the delegates vote



10) In this final screenshot we can see that after the delegation's vote the weight of the vote which was originally 1 (default) is now 3.



Conclusion:-

In this experiment, a Voting/Ballot smart contract was deployed using Solidity on the Remix IDE. The concepts of require statements, mapping, and data location specifiers like storage and memory were explored to understand their role in ensuring security, efficiency, and correctness in smart contracts. The difference between using bytes32 and string for proposal names was also studied, highlighting the trade-off between gas efficiency and readability. Overall, the experiment provided practical insights into the design and deployment of voting contracts on the blockchain.